

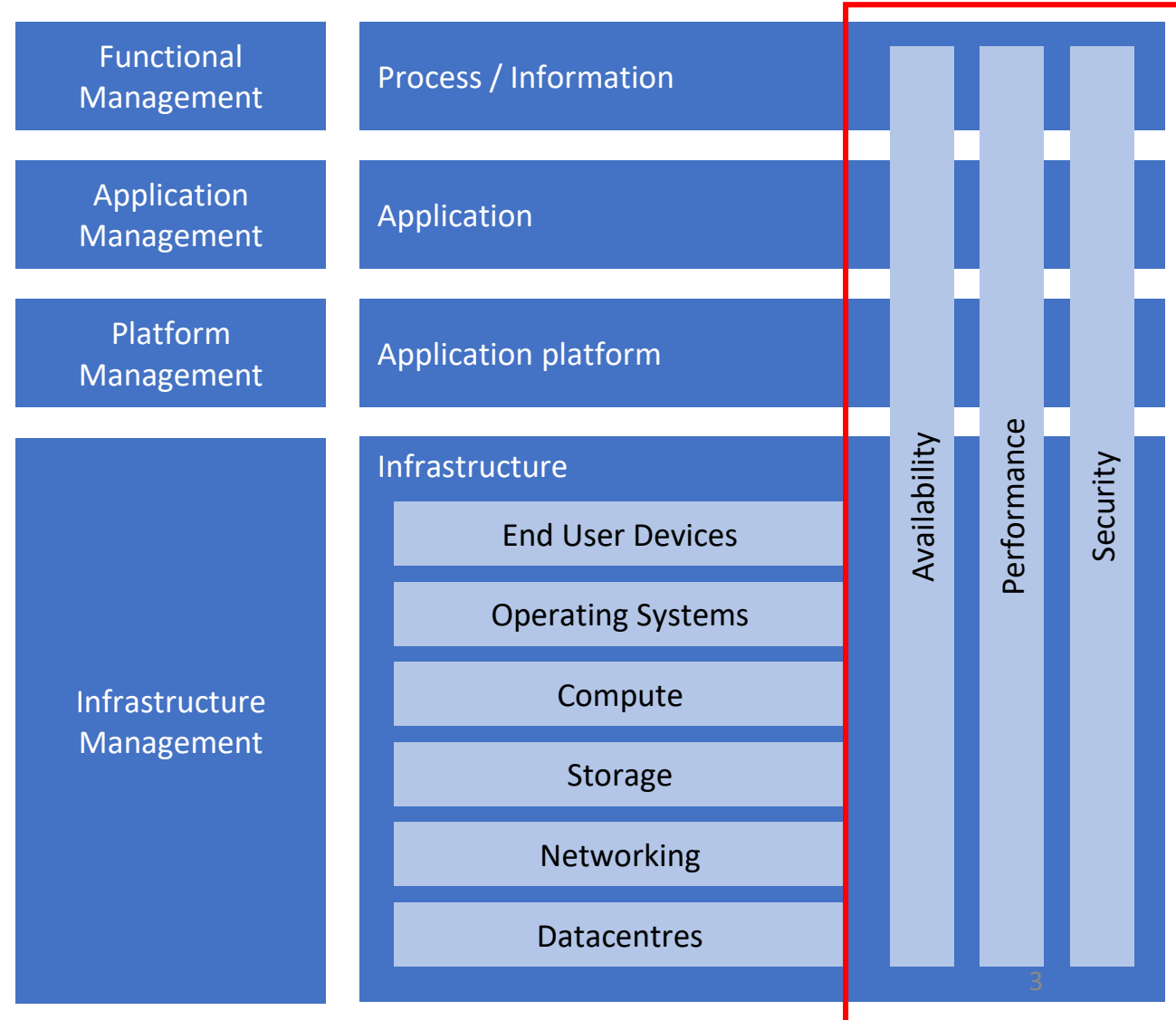
Chapter 2: Introduction to Non-functional attributes

Objectives

- Explain the concepts of Non-functional attributes
 - Availability
 - Performance
 - Security

Non-Functional attributes

- Non-functional attributes describe the **qualitative behaviour** of a system
- Essential in IT infrastructure architectures:
 - Availability
 - Performance
 - Security



NFRs

- The name “Non-functional attributes” suggests they **have no function**
- They are very important for the **successful implementation** and use of an IT infrastructure
- The term non-functional requirements or NFRs is frequently used and widely known
- The **acceptance of a system** is largely dependent on the implemented non-functional requirements

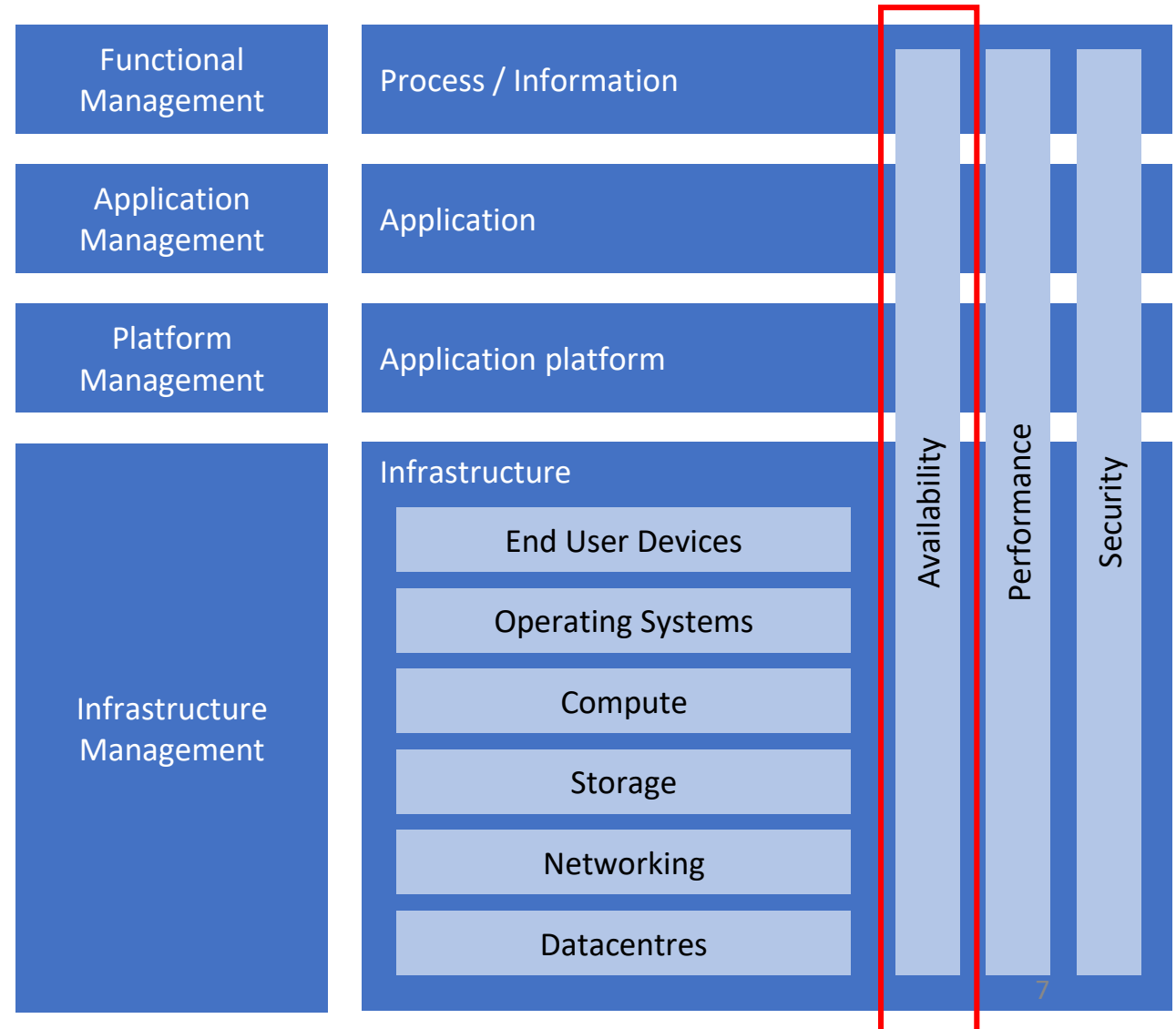
Conflicting NFRs

- Many of the non-functional attributes are delivered by the infrastructure
- Non-functional requirements are **often conflicting**:
 - Security versus user friendliness
 - Performance versus cost
- The infrastructure architect should present stakeholders with these conflicting requirements and their consequences

Availability

Introduction

- Everyone expects their infrastructure to be available all the time
- A 100% guaranteed availability of an infrastructure is impossible



Calculating availability

- Availability can **neither be calculated, nor guaranteed** upfront
 - It can only be reported on afterwards, when a system has run for some years
- Over the years, much knowledge and experience is gained on how to design high available systems
 - Failover
 - Redundancy
 - Structured programming
 - Avoiding Single Points of Failures (SPOFs)
 - Implementing systems management

Calculating availability

- The availability of a system is usually expressed as a percentage of uptime in a given time period
 - Usually one year or one month
- Example for downtime expressed as a percentage per year:

Availability %	Downtime per year	Downtime per month	Downtime per week
99.8%	17.5 hours	86.2 minutes	20.2 minutes
99.9% ("three nines")	8.8 hours	43.2 minutes	10.1 minutes
99.99% ("four nines")	52.6 minutes	4.3 minutes	1.0 minutes
99.999% ("five nines")	5.3 minutes	25.9 seconds	6.1 seconds

Calculating availability

- Typical requirements used in service level agreements today are 99.8% or 99.9% availability per month for a full IT system
- The availability of the infrastructure must be much higher
 - Typically in the range of 99.99% or higher
- 99.999% uptime is also known as carrier grade availability
 - For one component
 - Higher availability levels for a complete system are very uncommon, as they are almost impossible to reach

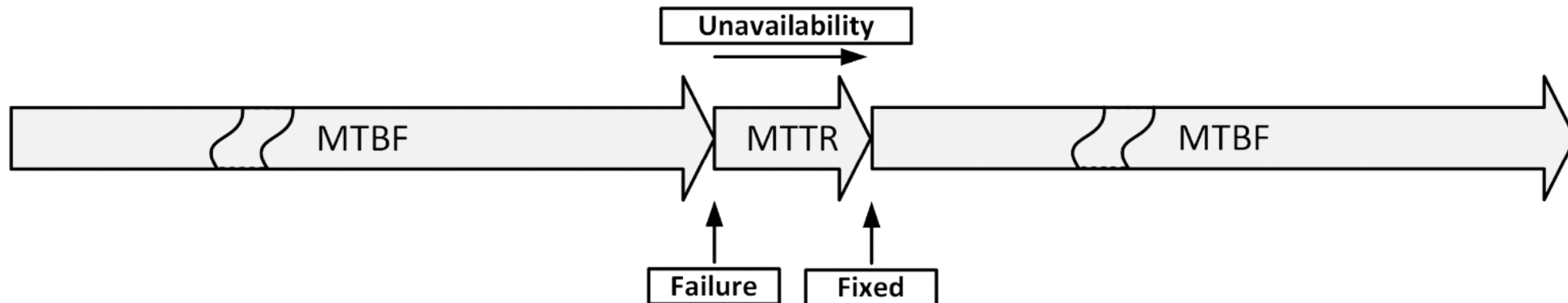
Calculating availability

- It is good practice to agree on the maximum frequency of unavailability

Unavailability (minutes)	Number of events (per year)
0 – 5	≤ 35
5 – 10	≤ 10
10 – 20	≤ 5
20 – 30	≤ 2
> 30	≤ 1

MTBF and MTTR

- Mean Time Between Failures (MTBF)
 - The average time that passes between failures
- Mean Time To Repair (MTTR)
 - The time it takes to recover from a failure



MTBF and MTTR

- Some components have higher MTBF than others
- Some typical MTB's:

Component	MTBF (hours)
Hard disk	750,000
Power supply	100,000
Fan	100,000
Ethernet Network Switch	350,000
RAM	1,000,000

Why a fan has a shorter MTBF than a HDD?

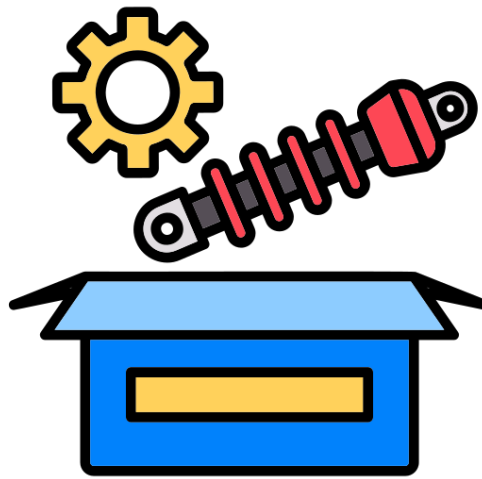


MTTR

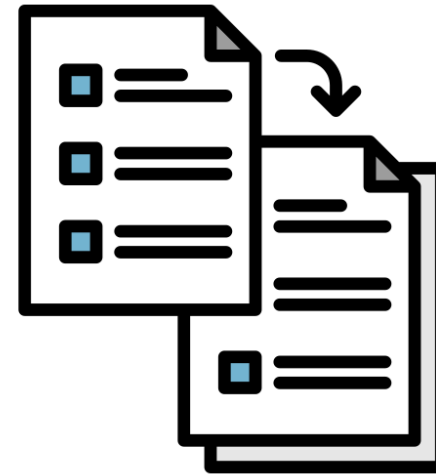
- MTTR can be kept low by:



Having a service contract with the supplier



Having spare parts on-site



Automated redundancy and failover

MTTR

- Steps to complete repairs:
 - Notification of the fault (time before seeing an alarm message)
 - Processing the alarm
 - Finding the root cause of the error
 - Looking up repair information
 - Getting spare components from storage
 - Having technician come to the datacenter with the spare component
 - Physically repairing the fault
 - Restarting and testing the component





Fault Summary



1



6



1



1

Equipment Servers LAN SAN VM Admin

Filter: Faults, Events and Audit ...

Faults, Events and Audit Log

- Faults
- Events
- Audit Logs
- Syslog
- Core Files
- TechSupport Files
- Settings

New Options Pending Activities Exit

>> Faults, Events and Audit Log

Faults, Events and Audit Log

Faults Events Audit Logs Syslog Core Files TechSupport Files Settings

Filter Export Print Hide Fault Details Show: ☐ All ☒ Category: ☒ All ☒ Generic ☒ Network ☒ Sysdebug ☒ Server ☒ FSM ☒ Management ☒ Operations ☒ Equipment ☒ Connectivity ☒ Configuration ☒ Environmental ☒ Security

Severity	Code	ID	Affected object	Cause
	F1000497	643849	policy-ep/scope-cont-managed-endpoint/context-sys_pki-ext/scope-pkiTP-name-*	fsm-failed

critical major minor warning info condition cleared flapping soaking suppressed

Save Changes

Reset Values

19

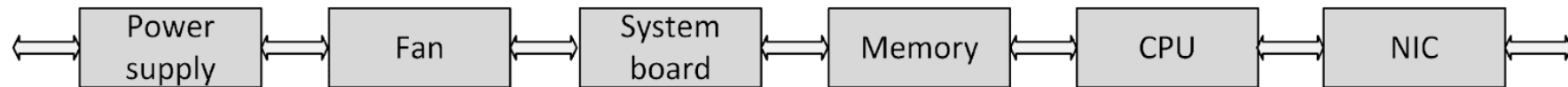
Calculation examples

•
$$\text{Availability} = \frac{\text{MTBF}}{(\text{MTBF} + \text{MTTR})} \times 100\%$$

Component	MTBF (h)	MTTR (h)	Availability	in %
Power supply	100,000	8	0.9999200	99.99200
Fan	100,000	8	0.9999200	99.99200
System board	300,000	8	0.9999733	99.99733
Memory	1,000,000	8	0.9999920	99.99920
CPU	500,000	8	0.9999840	99.99840
Network Interface Controller (NIC)	250,000	8	0.9999680	99.99680

Calculation examples

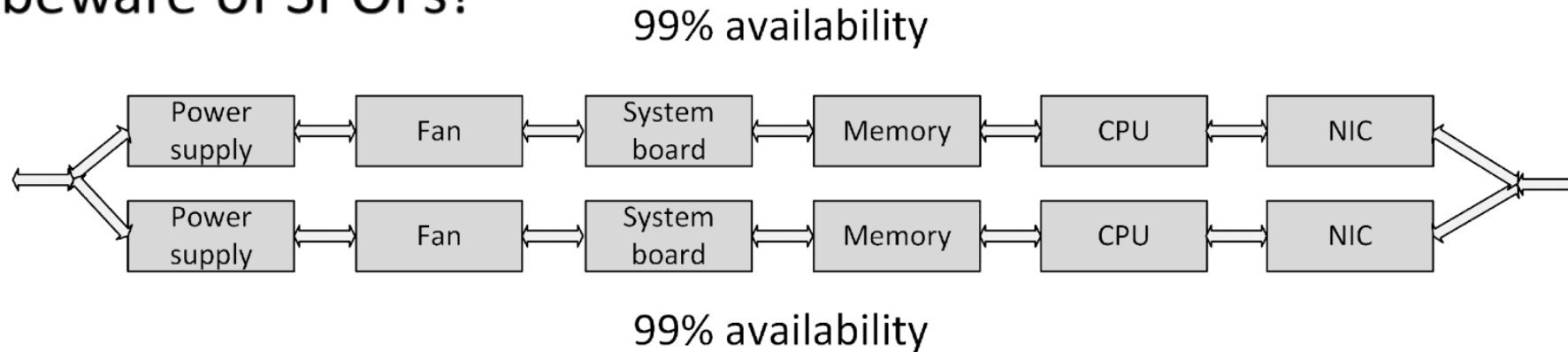
- Serial components: One defect leads to downtime



- Example: the above system's availability is:
- $0.9999200 \times 0.9999200 \times 0.9999733 \times 0.9999920 \times 0.9999840 \times 0.9999680 = 0.99977 = \mathbf{99.977\%}$
- (each components' availability is at least 99.99%)

Calculation examples

- Parallel components: One defect: no downtime!
- But beware of SPOFs!



- Calculate availability: $A_T = 1 - \prod_{i=1}^n (1 - A_i)$
- Total availability = $1 - (1 - 0.9999) \times (1 - 0.999) = 0.9999999$ (or 99.99999%)

Sources of unavailability - human errors

- 80% of outages impacting mission-critical services is caused by people and process issues
- Examples:
 - Performing a test in the production environment
 - Switching off the wrong component for repair
 - Swapping a good working disk in a RAID set instead of the defective one
 - Restoring the wrong backup tape to production
 - Accidentally removing files
 - Mail folders, configuration files
 - Accidentally removing database entries
 - Drop table x instead of drop table y



Sources of unavailability - software bugs

- Because of the complexity of most software it is nearly impossible (and very costly) to create bug-free software
- Application software bugs can stop an entire system
- Operating systems are software too
 - Operating systems containing bugs can lead to corrupted file systems, network failures, or other sources of unavailability

Sources of unavailability - planned maintenance

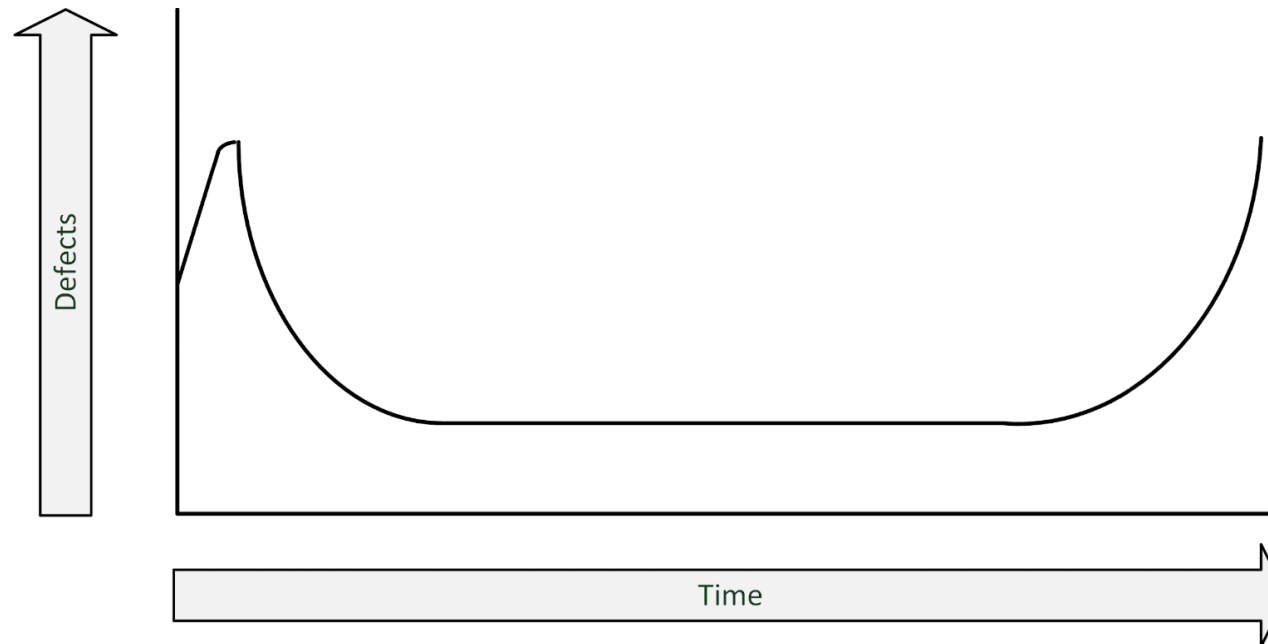
- Sometimes needed to perform systems management tasks:
 - Upgrading hardware or software
 - Implementing software changes
 - Migrating data
 - Creation of backups
- Should only be performed on parts of the infrastructure where other parts keep serving clients
- During planned maintenance the system is more vulnerable to downtime than under normal circumstances
 - A temporary SPOF could be introduced
 - Systems managers could make mistakes

Sources of unavailability - physical defects

- Everything breaks down eventually
- Mechanical parts are most likely to break first
- Examples:
 - Fans for cooling equipment usually break because of dust in the bearings
 - Disk drives contain moving parts
 - Tapes are very vulnerable to defects as the tape is spun on and off the reels all the time
 - Tape drives contain very sensitive pieces of mechanics that can break easily

Sources of unavailability - bathtub curve

- A component **failure** is most likely when the component is **new**
- When a component still works after the **first month**, it is likely that it will continue working without failure until the end of its life



Sources of unavailability - environmental issues

- Environmental issues can cause downtime:
 - Failing facilities
 - Power
 - Cooling
 - Disasters
 - Fire
 - Earthquakes
 - Flooding

Sources of unavailability - complexity of the infrastructure

- Adding more components to an overall system design can undermine high availability
 - Even if the extra components are implemented to achieve high availability
- Complex systems
 - Have more potential points of failure
 - Are more difficult to implement correctly
 - Are harder to manage
- Sometimes it is better to just have an extra spare system in the closet than to use complex redundant systems

Redundancy

- Redundancy is the duplication of critical components in a single system, to avoid a **single point of failure** (SPOF)
- Examples:
 - A single component having two power supplies; if one fails, the other takes over
 - Dual networking interfaces
 - Redundant cabling

**Application clients
(end users)**



**Internet
router**



**Load
balancer**



**Single point of
failure (SPOF)**

**Application
servers**



Failover

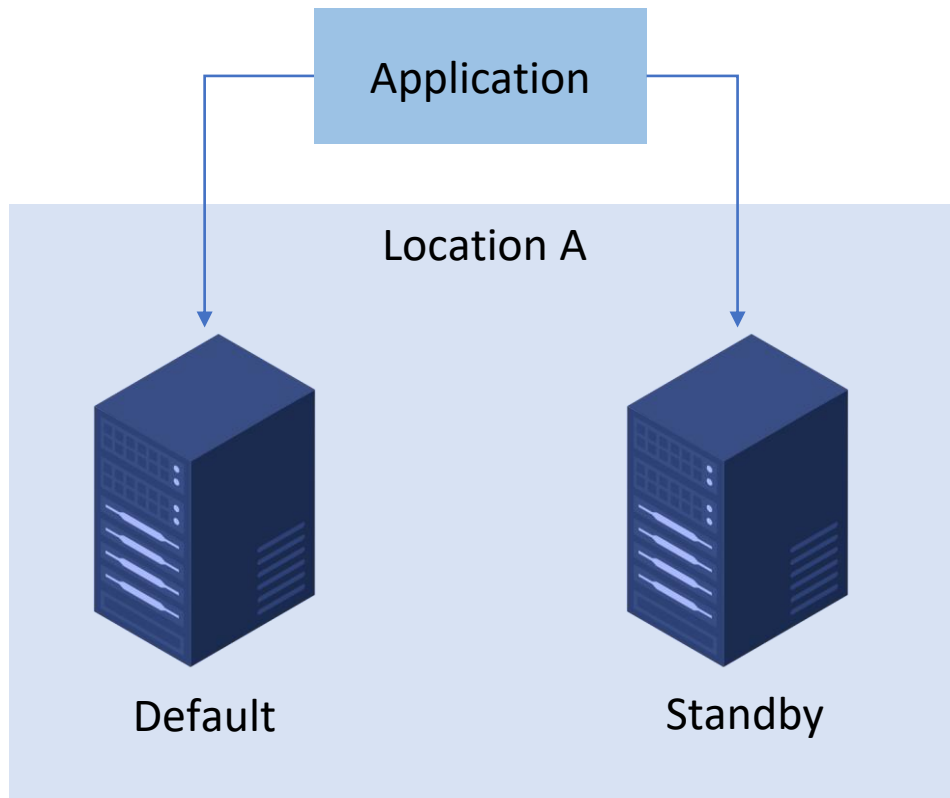
- Failover is the (semi)**automatic** switch-over to a **standby system or component**
- Examples:
 - Windows Server failover clustering
 - VMware High Availability
 - Oracle Real Application Cluster (RAC) database

Fallback

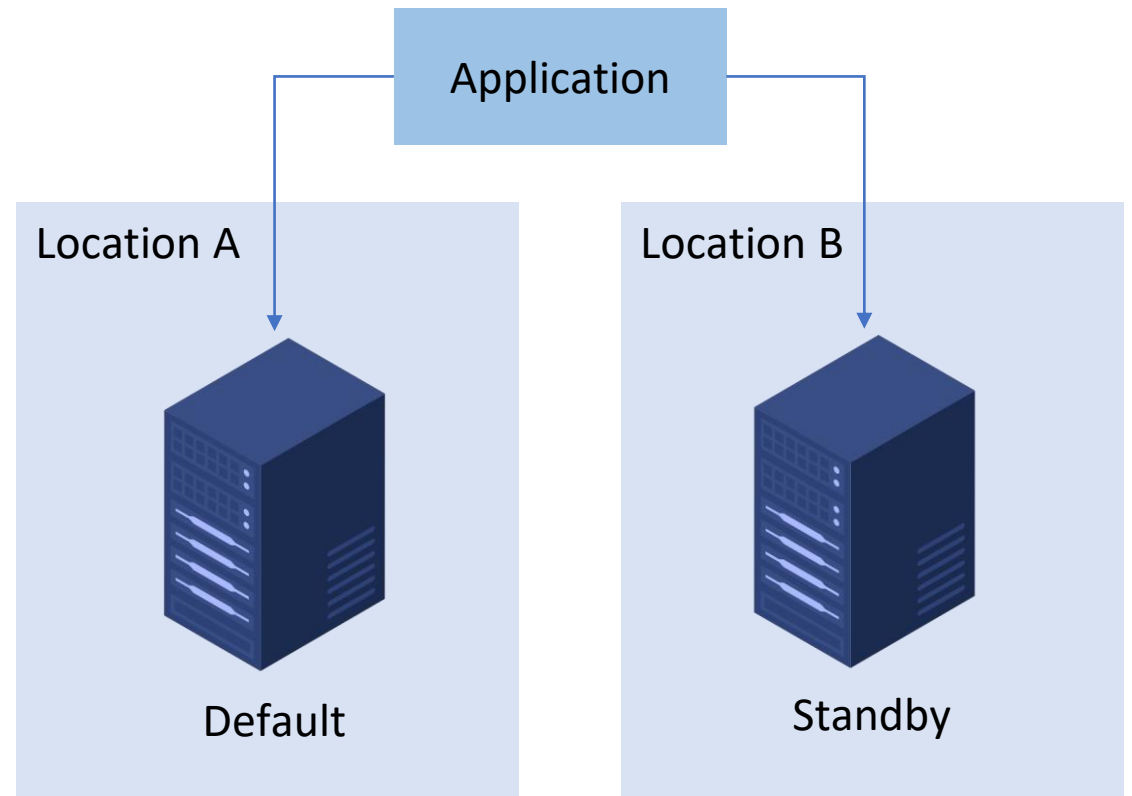
- Fallback is the **manual** switchover to an identical standby computer system in a **different location**
- Typically used for disaster recovery
- Three basic forms of fallback solutions:
 - Hot site
 - Cold site
 - Warm site

Failover vs Fallback

Failover



Fallback



Fallback – hot site

- A hot site is
 - A fully configured fallback datacentre
 - Fully equipped with power and cooling
 - Applications are installed on the servers
 - Data is kept up-to-date to fully mirror the production system
- Requires constant maintenance of the hardware, software, data, and applications to be sure the site accurately mirrors the state of the production site at all times

Fallback - cold site

- Is ready for equipment to be brought in during an emergency, **but no computer hardware is available** at the site
- **Applications will need to be installed** and current data fully restored from backups
- If an organization has very little budget for a fallback site, a cold site may be better than nothing

Fallback - warm site

- A **computer facility readily** available with power, cooling, and computers, but the **applications may not be installed** or configured
- A mix between a hot site and cold site
- Applications and data must be restored from backup media and tested
 - This typically takes a day

Availability in the cloud

- Regions and availability zones
 - A public cloud region is a separate geographic area (like US-West or North Europe)
 - In each region, the cloud provider has multiple, isolated locations known as availability zones
 - An availability zone can be thought of as a (large) datacenter facility. Each availability zone can provide local failover
 - Availability zones in one region are far apart and isolated in terms of power supply, network connectivity and possible physical damage. The probability of two availability zones in a region being unavailable at the same time is very low

Availability in the cloud

- Update and fault domains
 - Local failover in an availability zone can be created using update domains and fault domains.
 - Fault domains define the group of virtual machines that share a common power source and network switch. If you have a system that needs to provide a local failover scenario, you should place the redundant components of the system in multiple fault domains
 - If the redundant components of a high available solution are placed in two or more update domains, it is guaranteed that updates (possibly leading to a reboot of systems) will not affect the availability of the full system.

Business Continuity

- An IT disaster is defined as an irreparable problem in a datacenter, making the datacenter unusable
- Natural disasters: Floods, hurricanes, tornadoes, earthquakes
- Manmade disasters: Hazardous material spills, infrastructure failure, bio-terrorism

Business Continuity

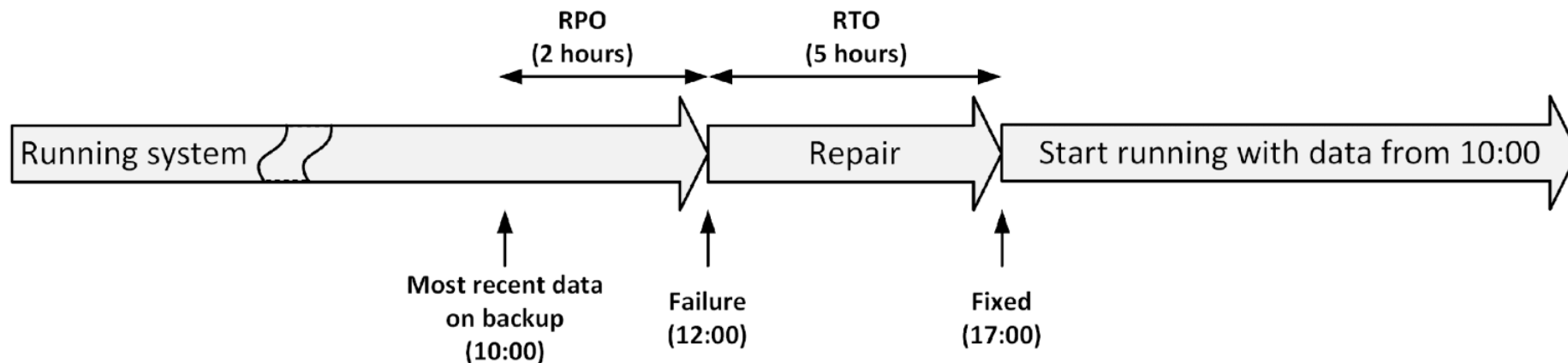
- In case of a disaster, the infrastructure could become unavailable, in some cases for a longer period of time
- Business Continuity Management includes:
 - IT
 - Managing business processes
 - Availability of people and work places in disaster situations
- **Disaster recovery planning** (DRP) contains a set of measures to take in case of a disaster, when (parts of) the IT infrastructure must be accommodated in an alternative location

RTO and RPO

- RTO and RPO are objectives in case of a disaster
- Recovery Time Objective (RTO)
 - The **maximum duration of time** within which a business process must be **restored after a disaster**, in order to **avoid unacceptable consequences** (like bankruptcy)

RTO and RPO

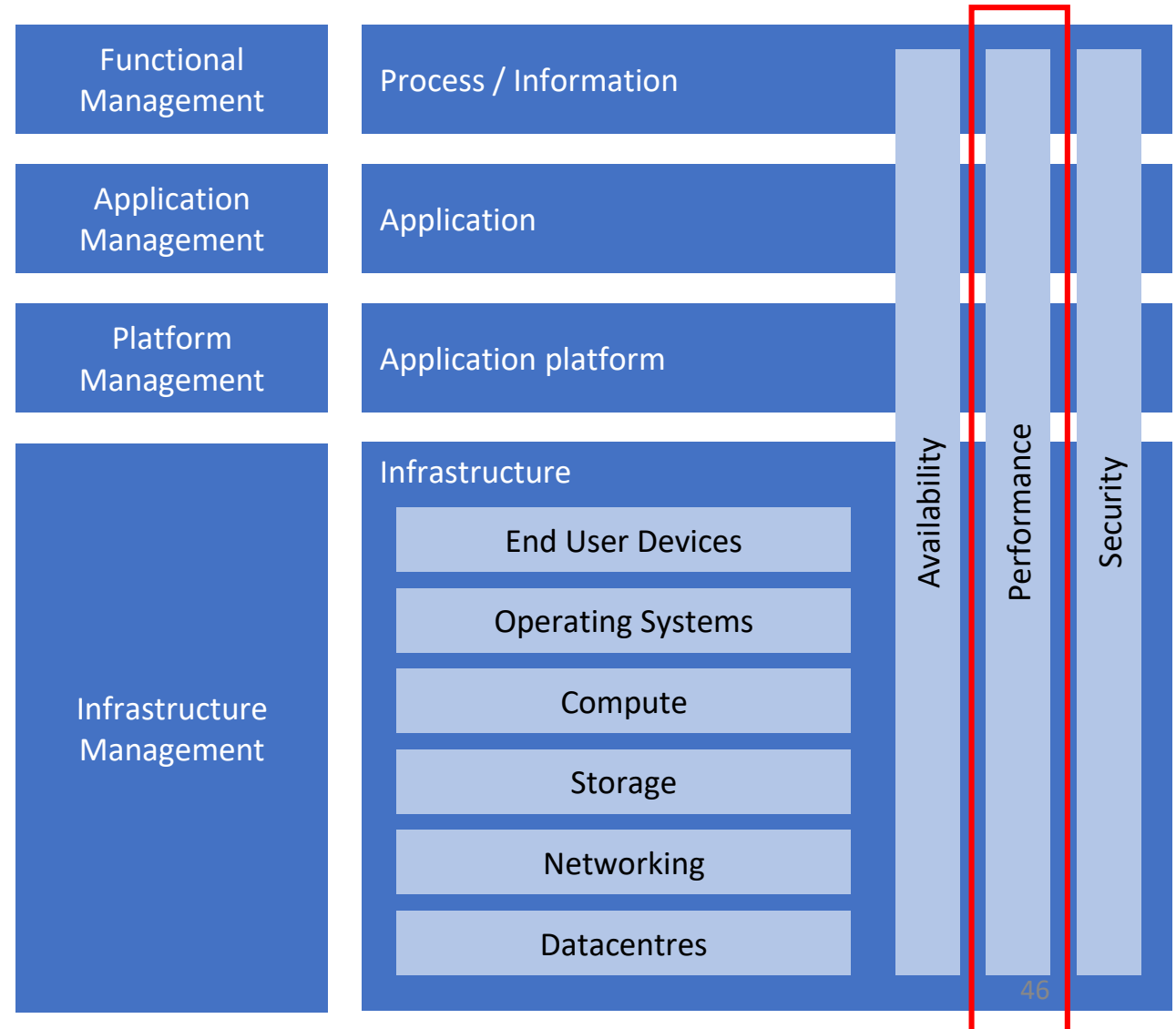
- Recovery Point Objective (RPO)
 - The point in time to which **data must be recovered** considering some "**acceptable loss**" in a disaster situation
- RTO and RPO are individual objectives
 - They are not related



Performance

Introduction

- Performance is a typical **hygiene factor**
- Nobody notices a highly performing system
- But when a system is not performing well enough, users quickly start complaining

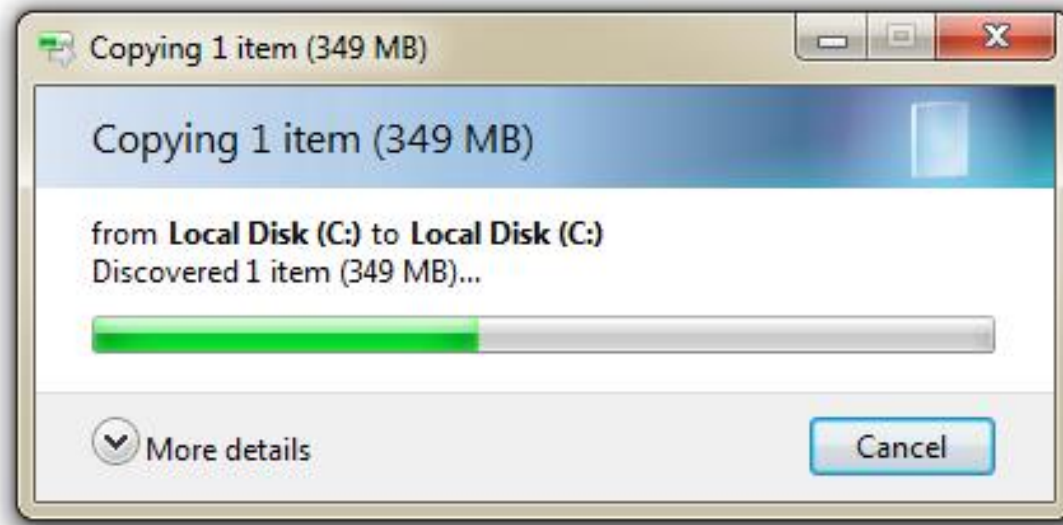


Perceived performance

- Perceived performance refers to **how quickly** a system appears to perform its task
- In general, people tend to overestimate their own patience
- People tend to value **predictability in performance**

Perceived performance

- Inform the user about how long a task will take
 - Progress bars
 - Splash screens



Performance during infrastructure design

- A solution must be designed, implemented, and supported to meet the **performance requirements even under increasing load**
- Calculating performance of a system in the design phase is:
 - Extremely difficult
 - Very unreliable

Performance during infrastructure design

- Performance must be considered:
 - When the system works as **expected** - normal
 - When the system is in a **special state**, like: failing parts, maintenance state, performing backup, and running batch jobs
- Some ways to do this are:
 - Benchmarking
 - Using vendor experience
 - Prototyping
 - User Profiling

Benchmarking

- A benchmark uses a specific test program to assess the relative performance of an infrastructure component
- Benchmarks compare:
 - Performance of various subsystems
 - Across different system architectures

Benchmarking

- Benchmarks comparing the raw speed of parts of an infrastructure
 - Like the speed difference between processors or between disk drives
 - Not taking into account the typical usage of such components
- Examples:
 - Floating Point Operations Per Second – FLOPS
 - Million Instructions Per Second – MIPS

Vendor experience

- The best way to determine the performance of a system in the design phase: use the experience of vendors
- They have a lot of experience running their products in various infrastructure configurations
- Vendors can provide:
 - Tools
 - Figures
 - Best practices

Prototyping

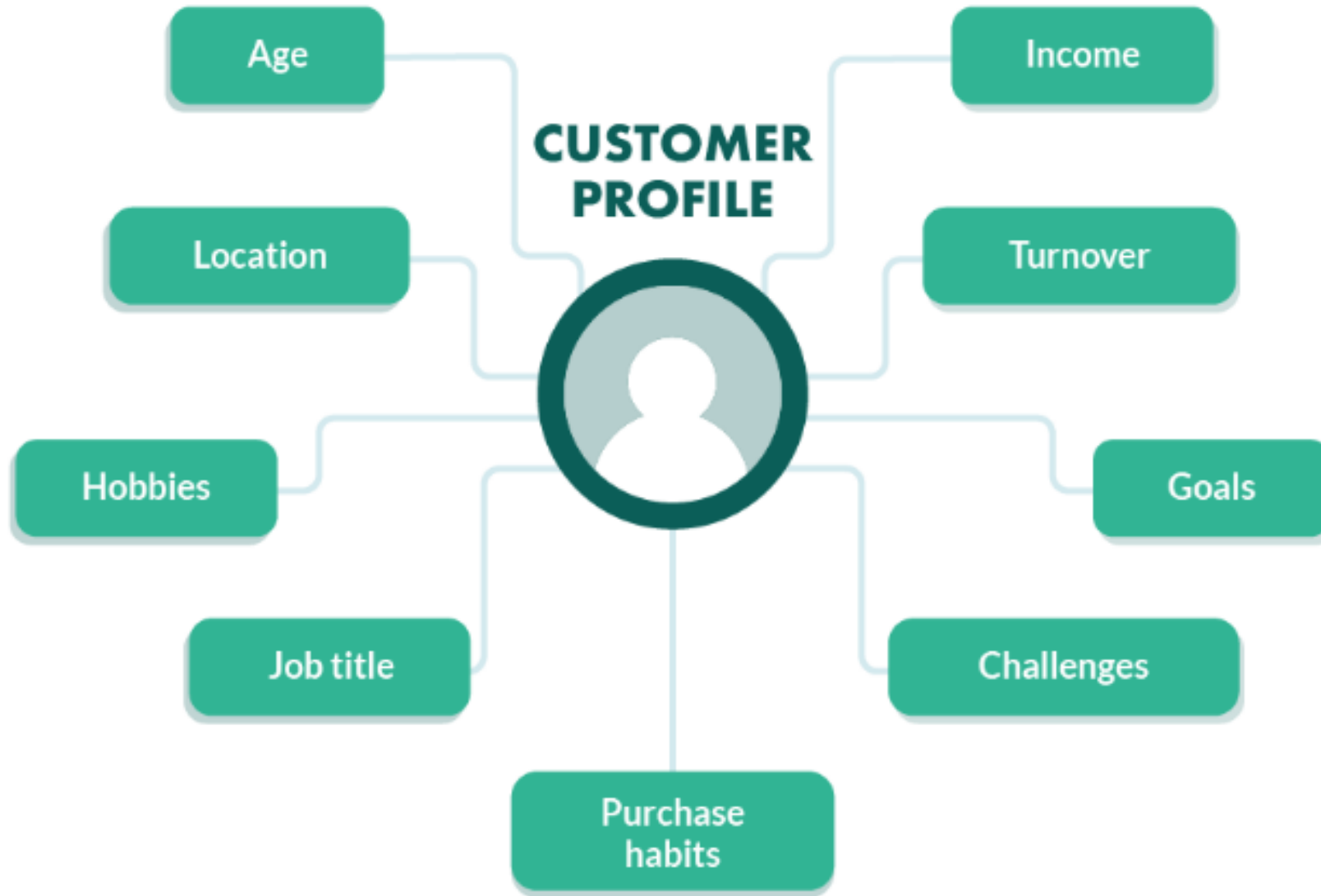
- Also known as proof of concept (PoC)
- Prototypes measure the performance of a system at an early stage
- Building prototypes:
 - Hiring equipment from suppliers
 - Using datacenter capacity at a vendor's premise
 - Using cloud computing resources
- Focus on those parts of the system that pose the highest risk, as early as possible in the design process

User profiling

- Predict the load a new software system will pose on the infrastructure before the software is actually built
- Get a good indication of the expected usage of the system
- Steps:
 - Define a number of typical user groups (personas)
 - Create a list of tasks personas will perform on the new system
 - Decompose tasks to infrastructure actions
 - Estimate the load per infrastructure action
 - Calculate the total load

User profiling personas/tasks

Persona	Number of users per persona	System task	Infrastructure load as a result of the system task	Frequency
Data entry officer	100	Start application	Read 100 MB data from SAN	Once a day
Data entry officer	100	Start application	Transport 100 MB data to workstation	Once a day
Data entry officer	100	Enter new data	Transport 50 KB data from workstation to server	40 per hour
Data entry officer	100	Enter new data	Store 50 KB data to SAN	40 per hour
Data entry officer	100	Change existing data	Read 50 KB data from SAN	10 per hour



User profiling Infrastructure load

Infrastructure load	Per day	Per second
Data transport from server to workstation (KB)	10,400,000	361.1
Data transport from workstation to server (KB)	2,050,000	71.2
Data read from SAN (KB)	10,400,000	361.1
Data written to SAN (KB)	2,050,000	71.2

Scalable cloud environments

- In cloud environments, there is often **no need to guess** the required **capacity**
- Simply **start with a large system** and **scale down** if the system is underutilized. Or vice versa: **Start with a small system** and **scale up** until the performance is acceptable
- Cloud environments usually have very extensive logging and monitoring capabilities to help with this

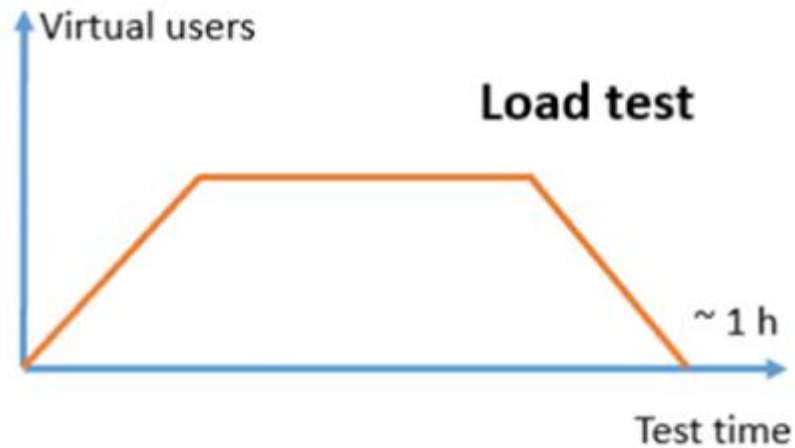
Managing bottlenecks

- The performance of a system is based on:
 - The performance of all its components
 - The interoperability of various components
- A component causing the system to reach some limit is referred to as the bottleneck of the system
- Every system has at least one bottleneck that limits its performance
- If the bottleneck does not negatively influence performance of the complete system under the highest expected load, it is OK

Performance testing

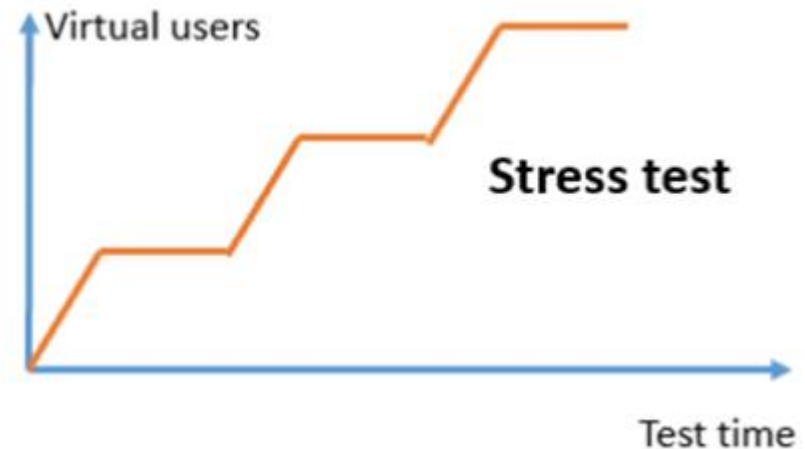
Load testing

- Shows how a system performs under the expected load



Stress testing

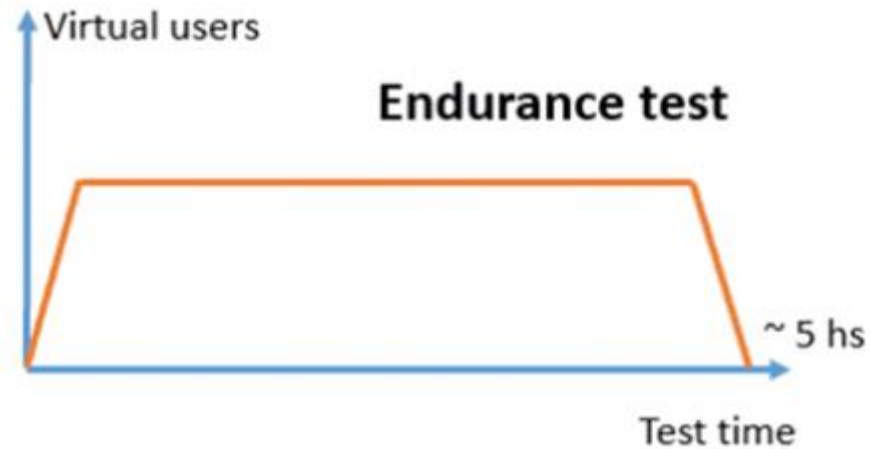
- Shows how a system reacts when it is under extreme load



Performance testing

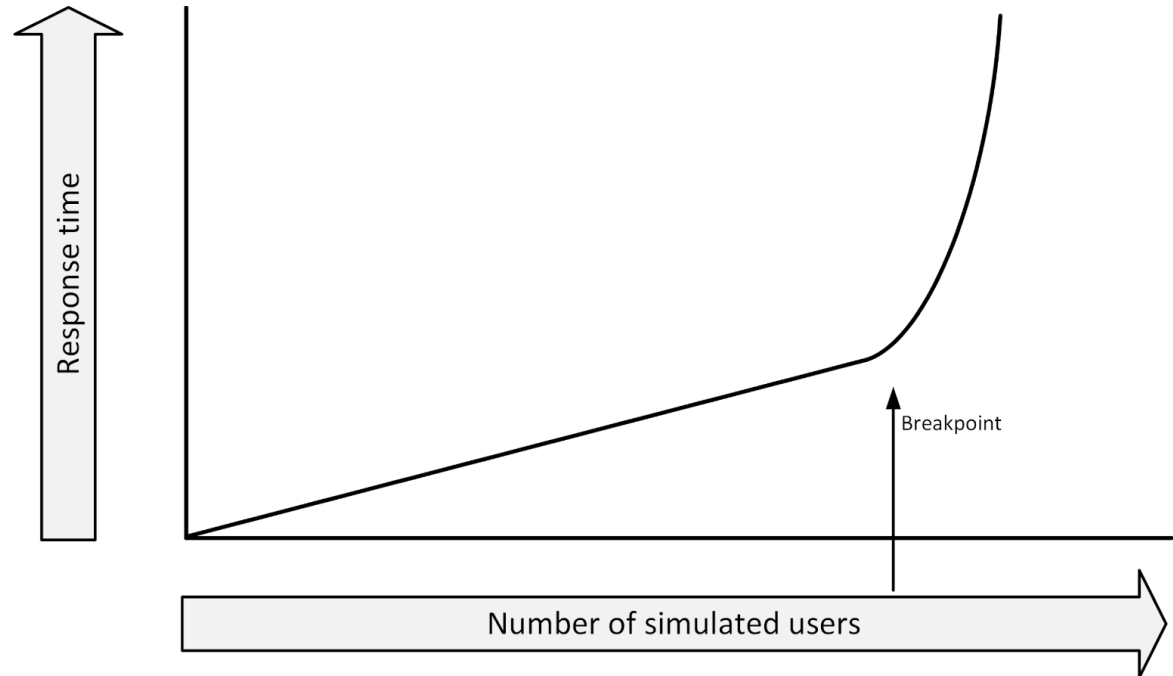
Endurance testing

- Shows how a system behaves when it is used at the expected load for a long period of time



Performance testing - Breakpoint

- Ramp up the load
 - Start with a small number of virtual users
 - Increase the number over a period of time
- The test result shows how the performance varies with the load, given as number of users versus response time.



Performance testing - Breakpoint

- A cloud environment is ideal for setting up performance testing environments
 - It can be scaled up to provide the required load and then scaled down after the test is complete
 - It can reduce the cost of running a performance test while simulating a very large number of users during the test period

Performance testing

- Performance testing software typically uses:
 - One or more servers to act as injectors
 - Each emulating a number of users
 - Each running a sequence of interactions
 - A test conductor
 - Coordinating tasks
 - Gathering metrics from each of the injectors
 - Collecting performance data for reporting purposes

Performance testing

- Performance testing should be done in a production-like environment
 - Performance tests in a development environment usually lead to results that are highly unreliable
 - Even when underpowered test systems perform well enough to get good test results, the faster production system could show performance issues that did not occur in the tests
- To reduce cost:
 - Use a temporary (hired) test environment

Increasing performance on upper layers

- 80% of the performance issues are due to badly behaving applications
- Application performance can benefit from:
 - Database and application tuning
 - Prioritizing tasks
 - Working from memory as much as possible (as opposed to working with data on disk)
 - Making good use of queues and schedulers
- Typically more effective than adding compute power

Disk caching

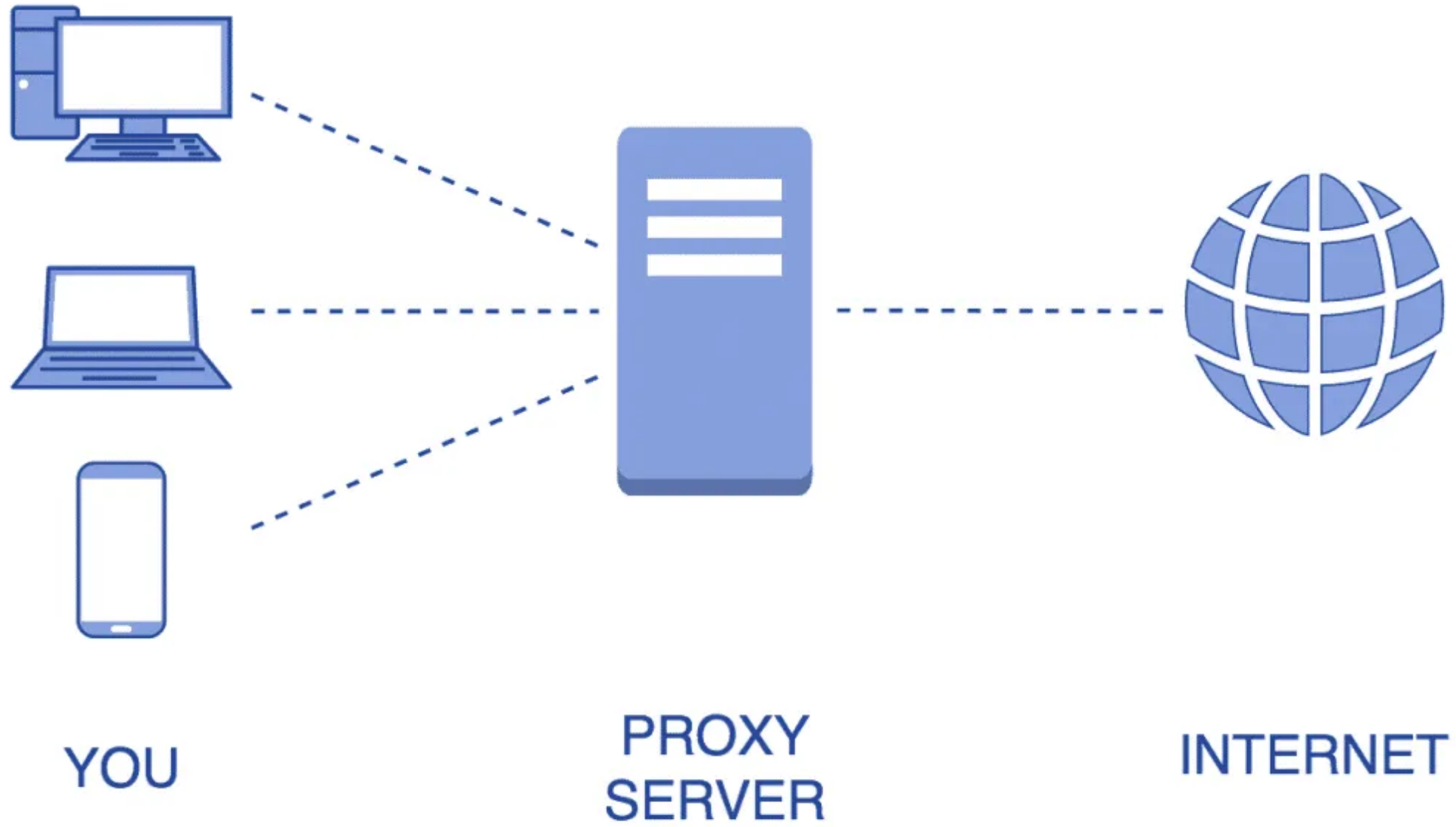
- Disks are mechanical devices that are slow by nature
- Caching can be implemented i:
 - Disks
 - Disk controllers
 - Operating system
 - All non-used memory in operating systems is used for disk cache
 - Over time, all memory gets filled with previously stored disk requests and prefetched disk blocks, speeding up applications.
- Cache memory:
 - Stores all data recently read from disk
 - Stores some of the disk blocks following the recently read disk blocks

Caching

Component	Time it takes to fetch <u>1 MB</u> of data (ms)
Network, 1 Gbit/s	675
Hard disk, 15k rpm, 4 KB disk blocks	105
Main memory DDR3 RAM	0.2
CPU L1 cache	0.016

Web proxies

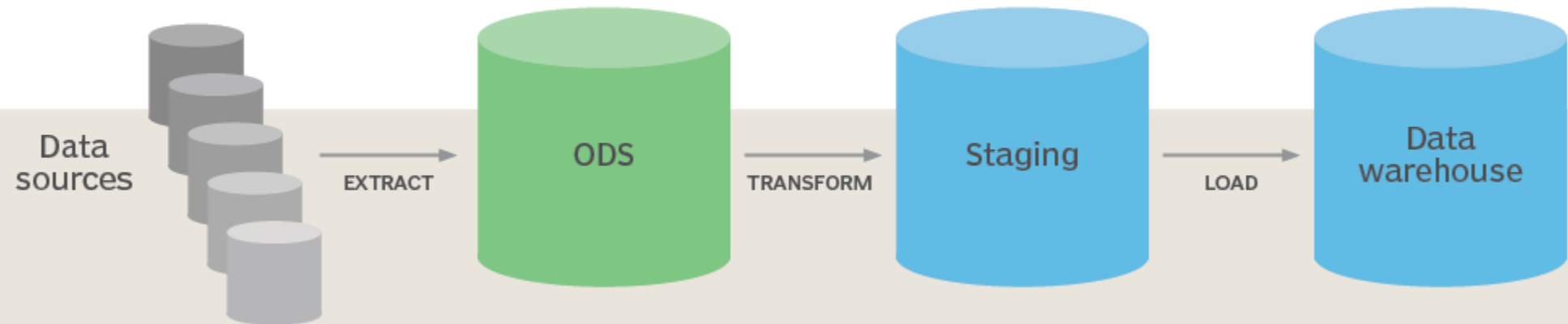
- When users browse the internet, data can be cached in a web proxy server
 - A web proxy server is a type of cache
 - Earlier accessed data can be fetched from cache, instead of from the internet
- Benefits:
 - Users get their data faster
 - All other users are provided more bandwidth to the internet, as the data does not have to be downloaded again



Operational data store

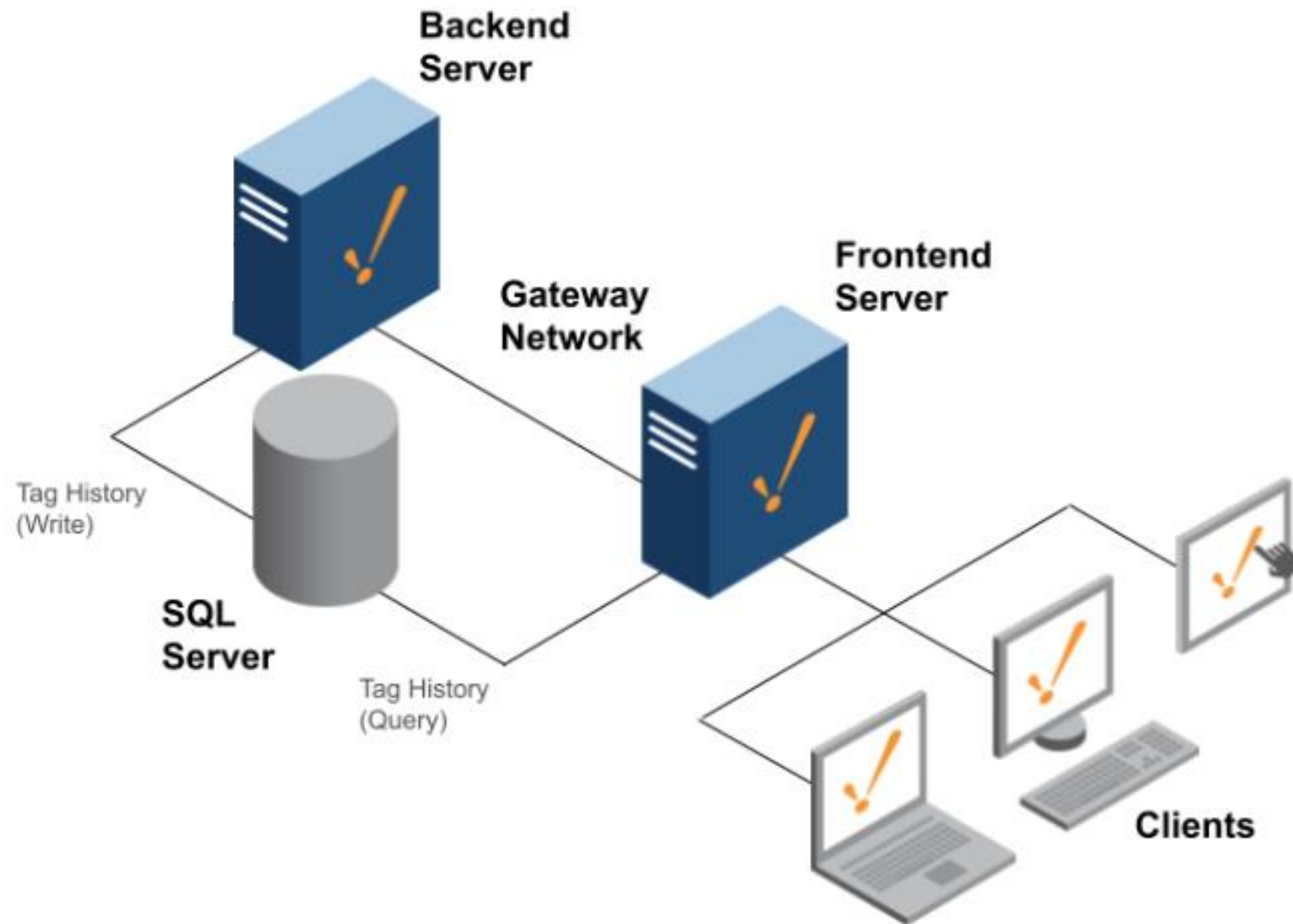
- An Operational Data Store (ODS) is a **read-only** replica of a part of a database, for a specific use
- Frequently used information is retrieved from a small ODS database
 - The main database is used less for retrieving information
 - The performance of the main database is not degraded

How an ODS works



Front-end servers

- Front-end servers serve data to end users
 - Typically web servers
- To increase performance, store static data on the front-end servers
 - Pictures are a good candidate
 - Significantly lowers the amount of traffic to back-end systems
- In addition, a reverse proxy can be used
 - Automatically cache most requested data



In-memory databases

- In special circumstances, entire databases can be run from memory instead of from disk
- In-memory databases are used in situations where performance is crucial
 - Real-time Supervisory control and data acquisition (SCADA) systems
 - High performance online transaction processing (OLTP) systems
 - As an example, in 2011 SAP AG introduced HANA, an in-memory database for SAP systems
- Special arrangements must be made to ensure data is not lost when a power failure occurs

Edge servers

- The major cloud providers have datacenters around the world
- In addition, they often offer edge locations. Edge locations can be used to cache data in close proximity to end users
- For example, a Web site that offers streaming videos can place copies of those videos in a number of edge locations around the world to ensure a good user experience for users in all locations

Scalability

- Scalability indicates the ease in with which a system can be modified, or components can be added, to handle increasing load
- Two ways to scale a system:
 - Vertical scaling (scale up) - adding resources to a single component
 - Horizontal scaling (scale out) - adding more components to the infrastructure

Vertical Scaling

(Improve or replace the existing server)



1 CPU
8 GB RAM
1 TB SSD



2 CPUs
16 GB RAM
3 TB SSD



4 CPUs
32 GB RAM
6 TB SSD

Horizontal Scaling

(Add more same-size nodes)

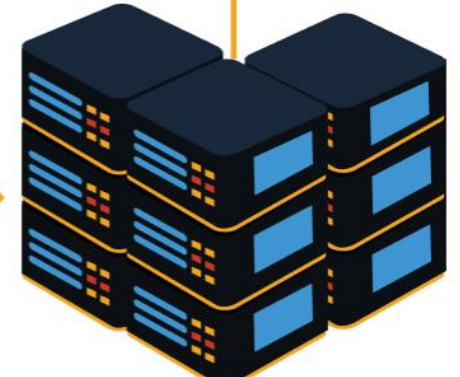
One server



1 CPU
8 GB RAM
1 TB SSD



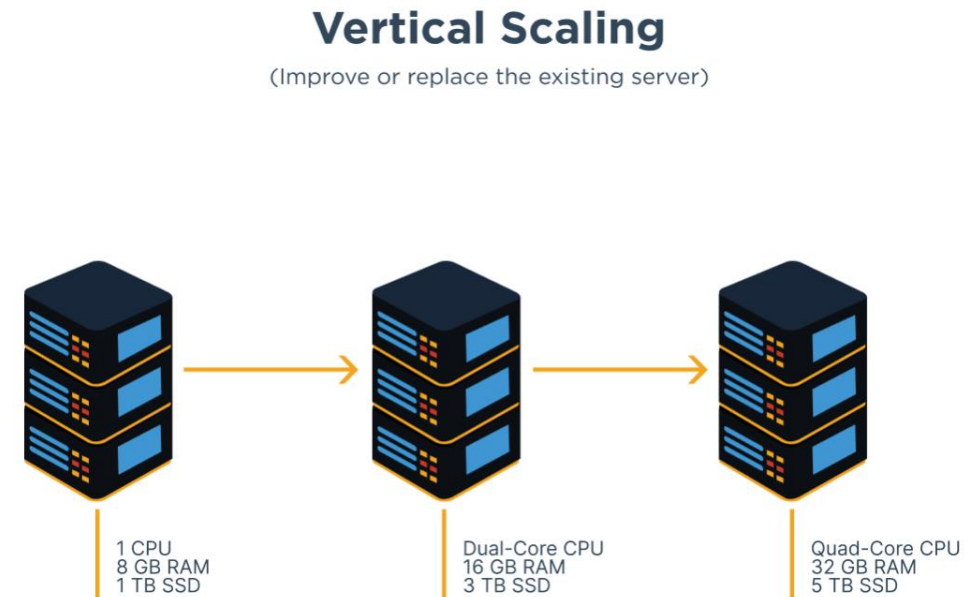
Pool of servers that
distribute the load



3 CPUs
24 GB RAM
3 TB SSD

Scalability – Vertical scaling

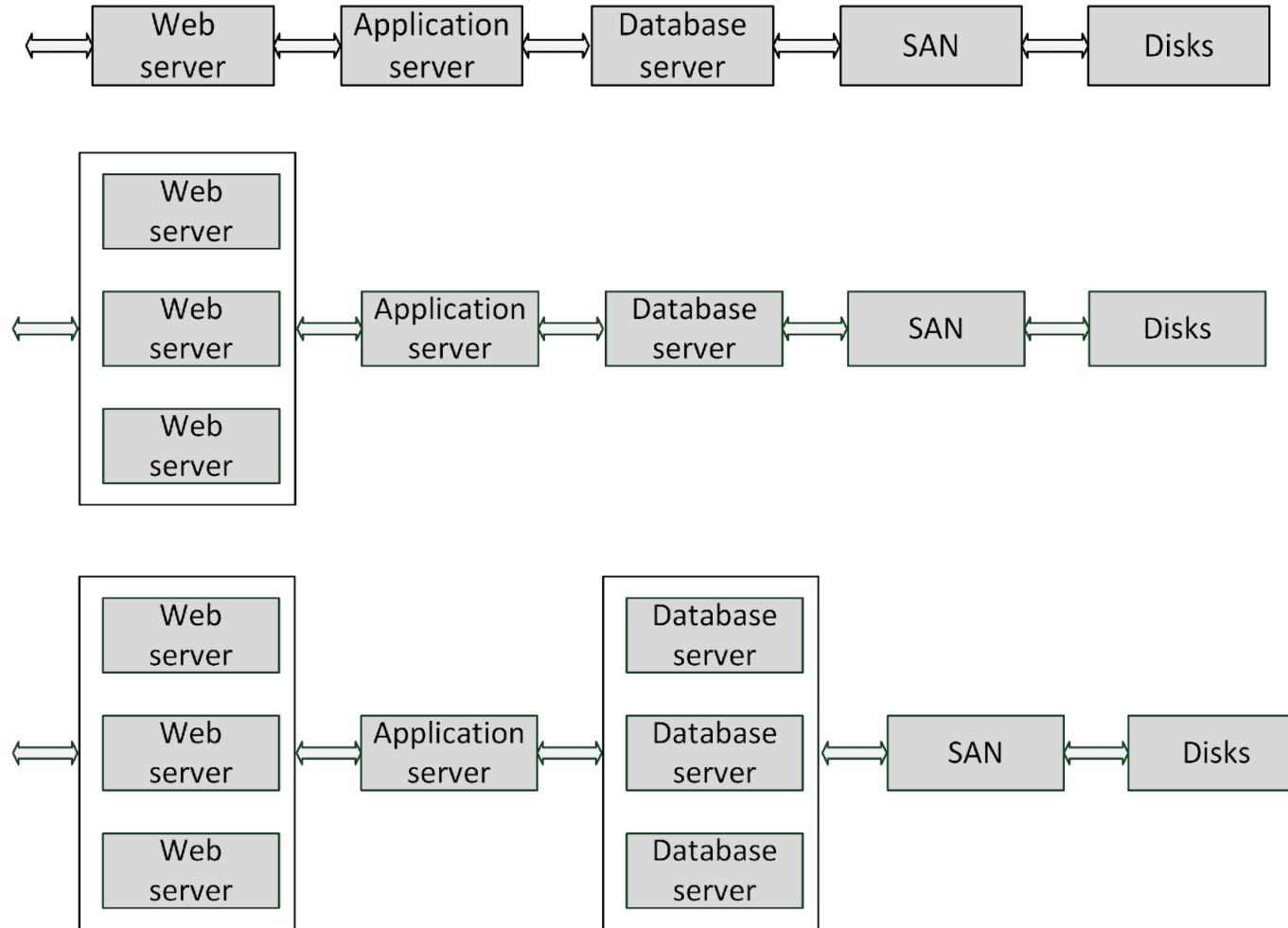
- Adding more resources, for example:
 - Server: more memory, CPU's
 - Network switch: adding more ports
 - Storage: Replace small disks by larger disks
- Vertical scaling is easy to do
- It quickly reaches a limit
 - The infrastructure component is “full”



Scalability – Horizontal scaling

- Adding more components to the infrastructure, for example:
 - Adding servers to a web server farm
 - Adding disk cabinets to a storage system
- In theory, horizontal scaling scales much better
 - Be aware of bottlenecks
- Doubling the number of components does not necessarily double the performance
- Horizontal scaling is the basis for cloud computing
- Applications must be aware of scaling infrastructure components

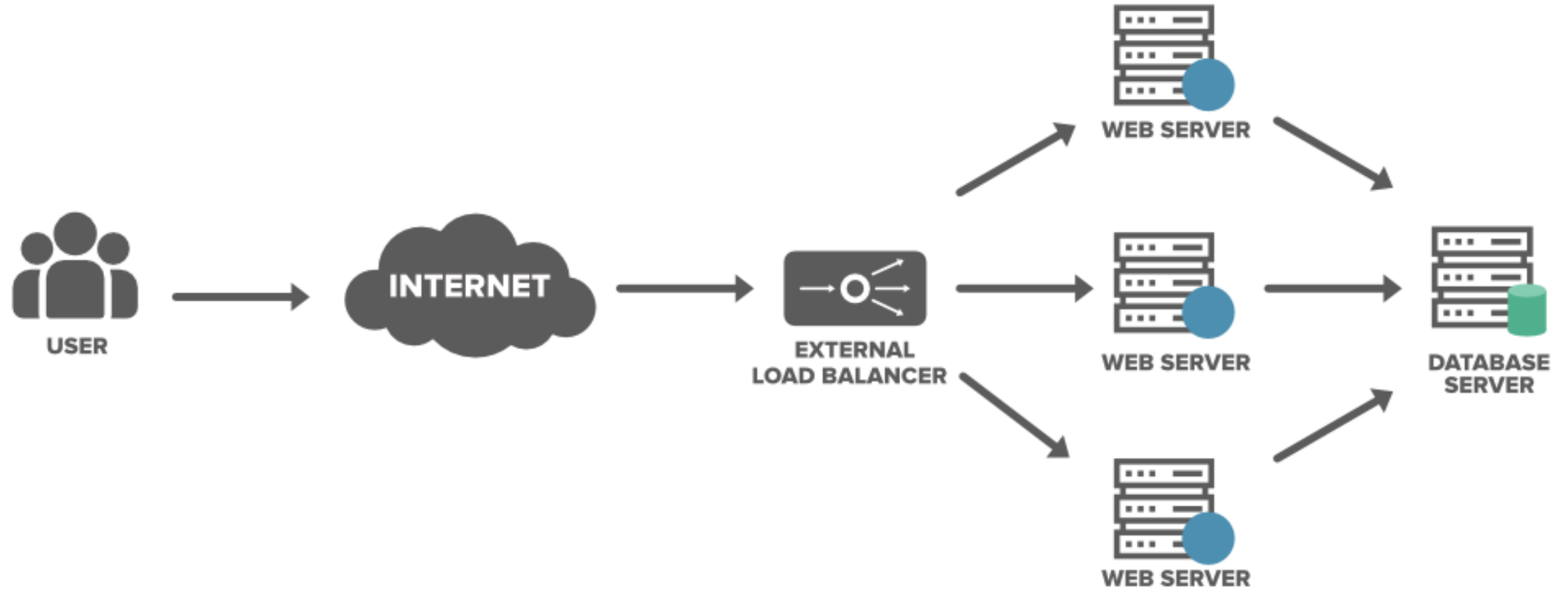
Scalability – Horizontal scaling



Load balancing

- Load balancing uses multiple servers that perform identical tasks
 - Examples:
 - Web server farm
 - Mail server farm
 - FTP (File Transfer Protocol) server farm
- A load balancer spreads the load over the available machines
 - Checks the current load on each server in the farm
 - Sends incoming requests to the least busy server

Load balancing



Load balancing

- Advanced load balancers can spread the load based on:
 - The number of connections a server has
 - The measured response time of a server
- The application running on a load balanced system must be able to cope with the fact that each request can be handled by a different server
 - The load balancer should contain the states of the application
 - The load balancing mechanism can arrange that a user's session is always connected to the same server
 - If a server in the server farm goes down, its session information becomes inaccessible and sessions are lost

Load balancing

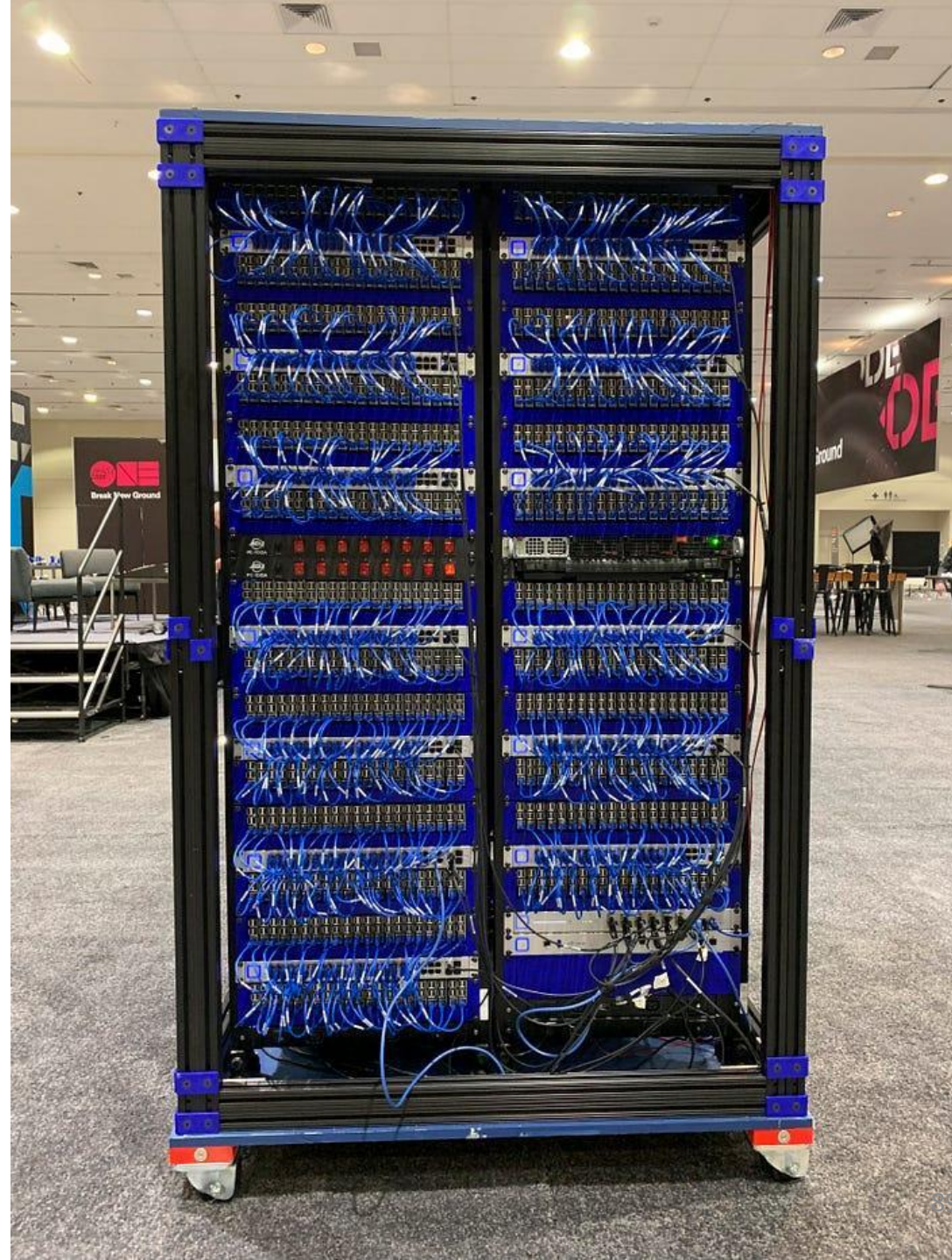
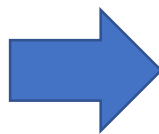
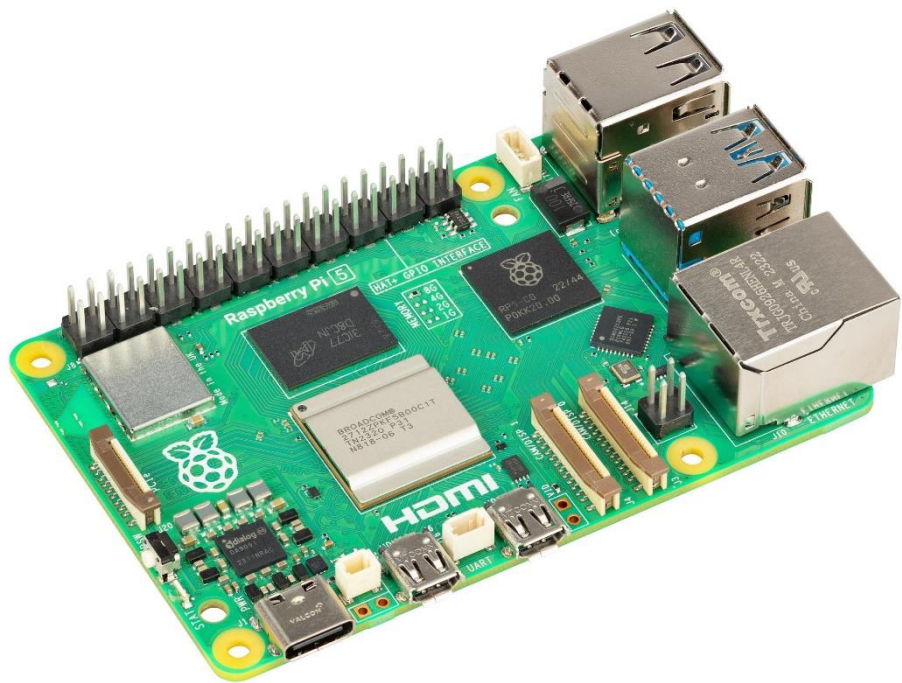
- A load balancer increases availability
 - When a server in the server farm is unavailable, the load balancer notices this and ensures no requests are sent to the unavailable server until it is back online again
- The availability of the load balancer itself is very important
 - Load balancers are typically setup in a failover configuration

Load balancing

- Network load balancing:
 - Spread network load over multiple network connections
 - Most network switches support port trunking
 - Multiple Ethernet connections are combined to get a virtual Ethernet connection providing higher throughput
 - The load is balanced over the connections by the network switch
- Storage load balancing:
 - Using multiple disks to spread the load of reads and writes
 - Use multiple connections between servers and storage systems

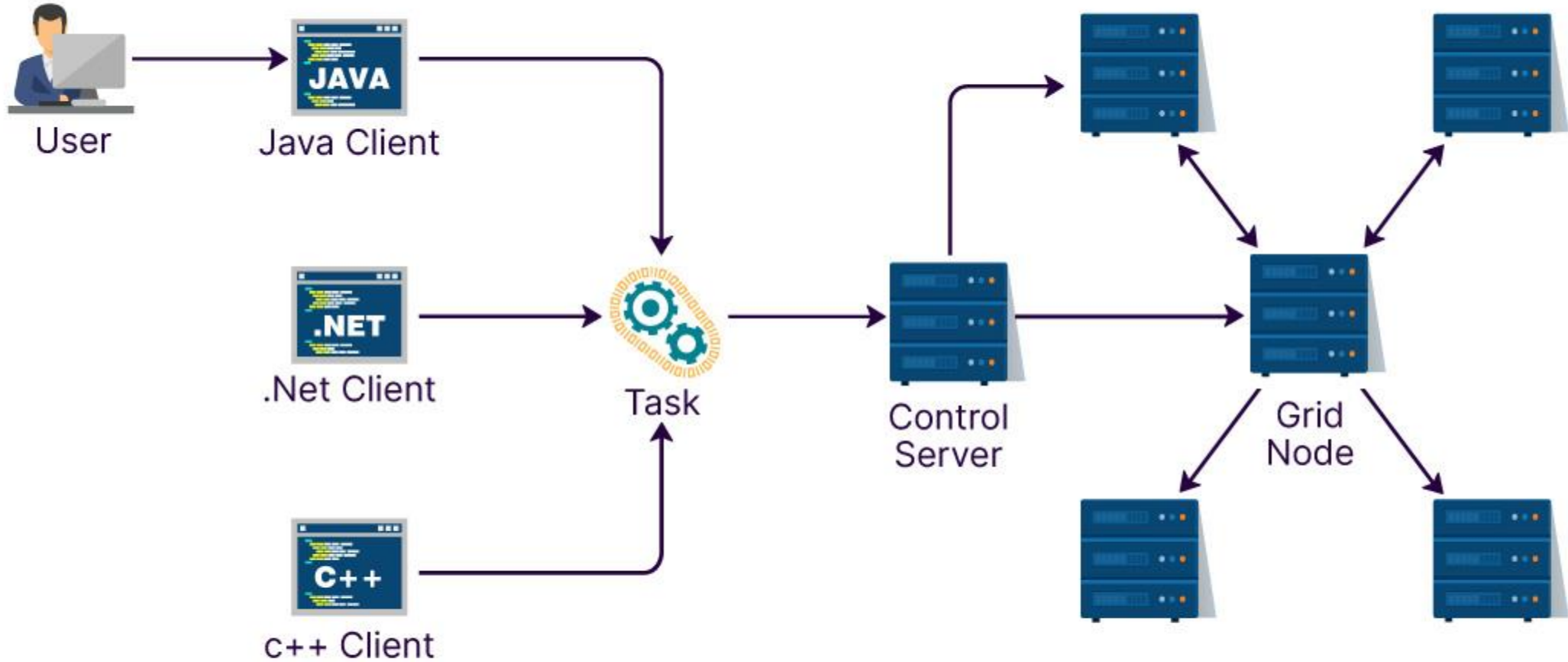
High performance clusters

- High performance clusters provide a vast amount of computing power by combining many computer systems
- A large number of cheap off the-shelf servers can create one large supercomputer
- Used for calculation-intensive systems
 - Weather forecasts
 - Geological research
 - Nuclear research
 - Pharmaceutical research
- TOP500.org



Grid Computing

- A computer grid is a high performance cluster that consists of systems that are spread geographically
- The limited bandwidth is the bottleneck
- Examples:
 - SETI@HOME
 - CERN LHC Computing Grid (140 computing centers in 35 countries)
- Broker firms exist for commercial exploitation of grids
- Security is a concern when computers in the grid are not under control



HOW GRID COMPUTING WORKS

Design for use

- Performance critical applications should be designed as such
- Tips:
 - Know what the system will be used for
 - A large data warehouse needs a different infrastructure design than an online transaction processing system or a web application
 - Interactive systems are different than batch oriented systems
 - When possible, try to spread the load of the system over the available time

Design for use

- In some cases, special products must be used for certain systems
 - Real-time operating systems
 - In-memory databases
 - Specially designed file systems
- Use standard implementation plans that are proven in practice
 - Follow the vendor's recommended implementation
 - Have the vendors check the design you created
- Move rarely used data from the main systems to other systems
 - Moving old data to a large historical database can speed up a smaller sized database

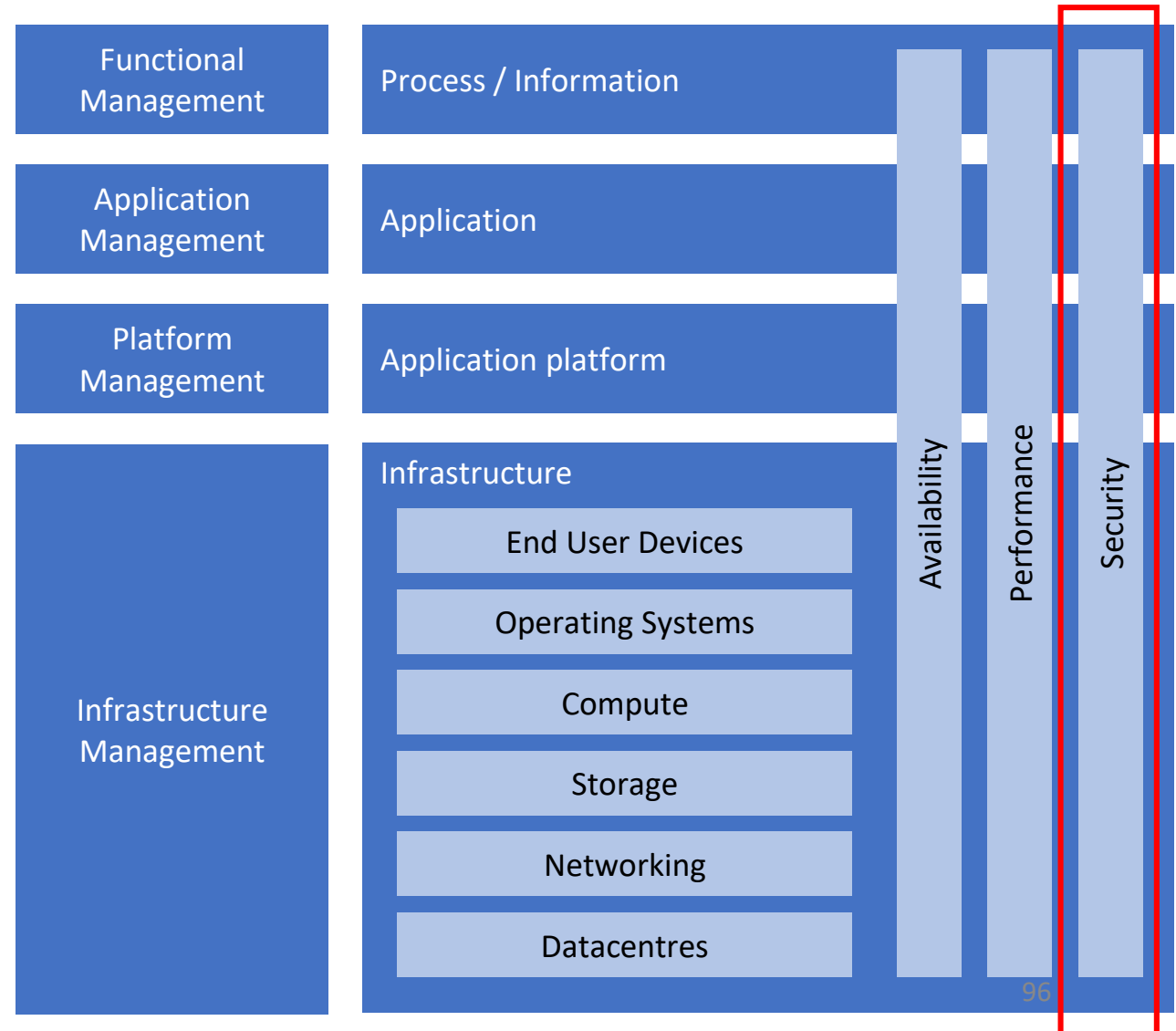
Capacity management

- Capacity management guarantees high performance of a system in the long term
- To ensure performance stays within acceptable limits, performance must be monitored
- Trend analyses can be used to predict performance degradation
- Anticipate on business changes (like forthcoming marketing campaigns)

Security

Introduction

- Security is the combination of:
 - Availability
 - Confidentiality
 - Integrity
- Focused on the recognition and resistance of attacks
- For IT infrastructures availability is a non-functional attribute in its own right



Core infrastructure security

Infrastructure security relies on three main technologies:

- Irreversibility of hash keys
- Practical unbreakable encryption
- Unbreachable virtualization

Crime against IT infrastructures

Motives for committing crime against IT infrastructures:

- Personal exposure and prestige
- Creating damage
- Financial gain
- Terrorism
- Warfare

Cloud security

- The public cloud follows a shared responsibility model
- The cloud provider takes care of security of the cloud
- The customer takes care of security in the cloud

	On-premises	IaaS	PaaS	SaaS
Data classification	Client	Client	Client	Client
Application configuration	Client	Client	Client	Client
Identity & Access	Client	Client	Client	Client
Application	Client	Client	Client	Cloud
Operating system	Client	Client	Cloud	Cloud
Compute	Client	Cloud	Cloud	Cloud
Storage	Client	Cloud	Cloud	Cloud
Network	Client	Cloud	Cloud	Cloud
Physical security	Client	Cloud	Cloud	Cloud

Cloud security

- The public cloud is very secure by default
- All components in the public cloud are designed with security in mind
- A lot is invested in security by the cloud providers and a large number of specialists work daily to optimize cloud security
- The cloud can easily become insecure through incorrect use or configuration errors

Prevention

- **Design for minimum risk.** Design the system to eliminate as much vulnerabilities as possible. This can for instance be done using source code analysis in software development and by running critical systems stand-alone instead of connected to other systems.
- **Incorporate safety devices.** Reduce risk using devices like firewalls and hardened screened routers. These devices usually don't affect the probability, but reduce the severity of an exploit: a firewall does not prevent attacks, but reduces the chance of an attacker connecting to sensitive parts of the network.
- **Implement training and procedures.** These can mitigate risks that are people-bound like social engineering attacks.
- Ensure that appropriate **security processes** are in place and regularly tested
- If possible, have security policies **monitored automatically**. Cloud providers in particular often provide options to enforce and monitor policies for all cloud resources in use.
- Use predefined policies, such as the **CIS benchmarks**

Implementation

- Zero trust
- Segregation of duties and least privilege
- Privileged Access Management (PAM)
- Layered security
- Identity and Access Management (IAM)
- Authentication
- Password
- Role Based Access Control (RBAC)
- Cryptography
- Encryption
- Computer Emergency Response Team (CERT)

Conclusion

- Explain the concepts of Non-functional attributes
 - Availability
 - Performance
 - Security